Pearls AQI Predictor

A Serverless System That Forecasts

3-Day Air Quality in Multan, Pakistan

Final Internship Report

Data Science Internship — 10Pearls Pakistan
Prepared by: Dawood Ahmad
Internship Duration: 8 weeks

Table of Contents

1. Executive Summary

During a 20-day internship at 10Pearls Pakistan, I built the Pearls AQI Predictor. It is a fully serverless system that forecasts Multan's Air Quality Index (AQI) three days ahead, at 24, 48, and 72 hours. The brief asked for the whole pipeline: getting the data, exploring it, building features, storing them, training models, retraining automatically, and showing results on a live dashboard. This report follows that same order — roughly how I actually built it.

Weather and air-quality data come from Open-Meteo. I turn that raw data into 47 features (time patterns, lag features, rolling averages), store them in a Hopsworks Feature Store, and train one Ridge Regression model for each forecast window. GitHub Actions keeps everything running on a schedule, and a Streamlit dashboard shows the forecasts to a user.

Ridge Regression beat both Random Forest and XGBoost. It didn't win right away — only after I checked results on a proper held-out test set instead of trusting cross-validation scores alone (Section 6.2 explains why that mattered). Here are the final results:

Horizon	RMSE	MAE	R^2
24h	22.36	17.56	0.785
48h	29.01	23.19	0.640
72h	30.45	24.68	0.604

Table 1. Final test results for each forecast horizon, measured on the most recent 13 months of data.

A lot of this report is also about things that went wrong at first: a data source that turned out to be unreliable, a testing setup that almost pointed me to the wrong model, and a Feature Store bug that took several days and a few false starts to fix. Section 9 covers all three in detail, because fixing them taught me more than the parts that worked the first time.

2. Introduction & Problem Statement

Multan's air quality changes a lot across the year. Winters are especially bad: crop burning plus a weather pattern called temperature inversion traps pollution close to the ground for weeks. People living with that — especially anyone with breathing problems — are better off knowing bad air is coming, rather than finding out by how the day feels. That's the problem this project solves: given the current hour, what will the AQI be one, two, and three days from now?

The internship brief set clear limits: everything had to run on free-tier tools, nothing could depend on a server I kept running myself, and the system had to go all the way from raw data to a dashboard a real person could check. That shaped the tools I used:

- Open-Meteo — free weather and air-quality data by location, no API key needed
- Hopsworks (free tier) — Feature Store and Model Registry
- GitHub Actions — runs the hourly feature pipeline and the daily training pipeline on schedule
- Streamlit Community Cloud — free hosting for the dashboard

The rest of this report follows the order I built things in: the data, the exploratory analysis, the features and feature store, the models, the automation, the dashboard, and finally the problems I ran into and what I learned from them.

3. Data Sources & Acquisition

3.1 Why I dropped AQICN

The original plan was to use AQICN/WAQI, as the brief suggested. It didn't work well for Multan. Looking up the city returned a monitoring station in Dushanbe, Tajikistan — a different country entirely — and the Multan station IDs I found by hand were often out of date or missing data. Rather than build a forecasting system on a data source I couldn't trust, I switched completely to Open-Meteo. It gives weather and air-quality data by map coordinates instead of by station, so it doesn't depend on any single sensor being online.

3.2 Final data source and feature set

Everything now comes from two Open-Meteo endpoints, fixed to Multan's coordinates (30.1575° N, 71.5249° E):

- Weather (archive-api.open-meteo.com/v1/archive, plus a forecast endpoint for current conditions): temperature_2m, relative_humidity_2m, wind_speed_10m, surface_pressure, cloud_cover, precipitation
- Air quality (air-quality-api.open-meteo.com/v1/air-quality): us_aqi, pm2_5, pm10, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone, dust

Two variables I tried early on didn't make it into the final set. Ammonia was 100% empty for these coordinates — there's no model coverage this far east — and pollen data only covers Europe, so it's not available for Pakistan at all. The two data feeds join cleanly on a shared time column, giving 15 raw columns. I used us_aqi (the US EPA scale) as the target the whole way through, instead of the European AQI scale the same API also offers.

3.3 Historical backfill

I loaded about three years of hourly history so the model would see a full year's cycle of seasons, not just one winter's smog. I rebuilt this backfill once, completely, partway through the project — Section 9.3 explains why.

4. Exploratory Data Analysis

4.1 Seasonal structure

Multan's air quality moves through three clear patterns across the year:

- Monsoon (August–September): moderate AQI — the cleanest part of the year
- Winter smog (October–January): severe and long-lasting highs, caused by crop burning combined with a temperature inversion that keeps pollution trapped near the ground
- Spring dispersal (February–May): AQI slowly improves as the inversion breaks up

Of all the weather variables, surface pressure had the strongest link to AQI — which makes sense, given how much the winter pattern depends on inversions. Among pollutants, PM2.5 was the clearest driver of overall AQI, which isn't surprising since the US EPA formula weights fine particles so heavily.

4.2 Outlier investigation

My first attempt at finding outliers used one threshold for the whole dataset, and it flagged a group of high-AQI hours in June. I didn't just accept that. I redid the check using a separate threshold for each month instead (the same method a monthly boxplot already uses), and got a clearer, more interesting picture:

- Winter's high outliers (Nov/Dec/Jan, 92 hours, average AQI 311, range 261–340) aren't separate events at all — they're just the extreme edge of an already bad season. I summarized these as a group instead of investigating each one.
- A spike on June 12–13 (10 hours, AQI up to 226, dust up to 482) held up as a real, one-off event: a dust storm, with nothing else like it in three years of data.
- Three low-AQI dips (Oct 14 2023, Oct 18 2023, Nov 11–12 2023) I first guessed were caused by strong wind clearing the air. That guess didn't hold up — wind speed was unremarkable during all three dips, and in one case it actually dropped to near zero right when AQI was at its lowest, the opposite of what a wind-driven story would predict. The real explanation is a normal daily pattern: each dip lines up with the hottest, driest part of the day and reverses overnight, as heat mixes pollution higher into the air during the day and cooling traps it low again at night. It's the same daily pattern already visible in the hour-by-hour AQI averages — just a bigger swing of it than usual.
- A fourth case (Feb 20 2024) didn't fit that pattern either — dust stayed high even as AQI dropped, along with a sharp temperature drop that looks like a cold front moving through. I've left this one as an unexplained edge case rather than force it into either story.

I'm including this whole process on purpose. Making a guess, testing it against the real data, and changing my mind when it didn't hold up felt like a more honest result than just confirming what I expected to find. It also directly shaped two later feature choices: the hour-of-day cyclical features, and the lag features that capture this same daily pattern.

5. Feature Engineering & Feature Store

5.1 Engineered features

Starting from the 15 merged raw columns, I built:

- Calendar features: hour, day, month, day_of_year, day_of_week
- Cyclical features: month_sin/month_cos, hour_sin/hour_cos, so that December and January are treated as close together instead of far apart
- Lag features (1h, 3h, 6h, 12h, 24h) for AQI, wind speed, and surface pressure
- Rolling stats: 6-hour and 24-hour rolling average AQI, and 24-hour rolling standard deviation
- Change-rate features: how much AQI changed in the last 1 hour and 6 hours
- Three targets, one per horizon: target_aqi_24h, target_aqi_48h, target_aqi_72h — each is just the AQI value shifted forward in time

That comes to 47 columns in total, which matches what's actually stored in the Hopsworks Feature Group.

5.2 Feature Store design (Hopsworks)

Features are stored in a Hopsworks Feature Group called aqi_features_multan, using time as both the primary key and the event-time column, with HUDI as the storage format. Two design choices are worth explaining in detail.

Self-healing targets

Since the three targets are just future AQI values, the newest rows in any given update can't have a real target yet — the future hasn't happened. Those rows go in with an empty (null) target. As the hourly pipeline keeps running and real values for those times become available, Hopsworks automatically replaces the empty target with the real number, using the shared primary key. No extra backfill step is needed. I also added a safety check on the training side: it drops any row with a still-empty target before splitting the data into train and test sets, so an unfinished row can never sneak into evaluation.

A data-quality bug I found and fixed

I found a bug where a cleanup step was supposed to remove incomplete rows, but the result of that step was never actually saved. Because of this, about 24 rows near the start of the backfill — rows that didn't have enough history yet to calculate a 24-hour lag — were stored with empty values instead of being removed like I intended. I fixed the notebook, and added two permanent checks on top of the fix: one that confirms the hourly timeline has no gaps (since a single missing hour would quietly throw off every lag feature after it, with no error shown), and one that checks the status of every Hopsworks save, since I'd earlier seen a save fail while the notebook kept going as if nothing was wrong.

Why there are two Feature Group versions

Anyone looking through the Hopsworks project will notice aqi_features_multan exists as both version 1 and version 2. This is simply how the project evolved, not a planned design. Version 1 is from an early stage, when I was only predicting a single 72-hour target and hadn't built the self-healing target design yet. Version 2 became the real, final version once I'd settled on predicting all three time windows. For a while, my training notebook was still pointed at the old version 1 by mistake — I caught this during a later review and fixed it before the final training run. Version 1 is kept in Hopsworks only as a record of that early stage; nothing in the finished system uses it.

6. Model Development & Evaluation

6.1 Train/test split

I split the data by date, not randomly: the most recent 13 months were used as the test set, and everything before that for training. I chose 13 months instead of an even 12 so the test period would cover a full year of seasons, rather than starting or ending partway through one.

6.2 Model comparison

I compared three types of models: Ridge Regression, Random Forest, and XGBoost. An LSTM (a type of neural network for sequences) was allowed by the brief, but I didn't build one — nothing in the data suggested it would beat the simpler model I already had, and building one properly wouldn't have fit in the time left.

All three models were tuned using GridSearchCV with 5-fold time-series cross-validation. That tuning step led to the most important discovery of the whole project.

Model	Test RMSE	Test R^2	Train R^2	Verdict
Ridge (tuned, α=10.0)	30.45	0.604	~0.60	Stable — CV and test agree
Random Forest (tuned)	33.31	0.526	0.931	Overfit — CV score was too optimistic
XGBoost (tuned)	33.42	0.523	0.998	Badly overfit — almost memorized the training data

Table 2. Final tuned model comparison on the 72-hour forecast (used as the example case).

The cross-validation trap

While tuning, Random Forest and XGBoost showed cross-validation scores (RMSE around 16 and 11) that looked much better than Ridge's. If I'd stopped there, I would have picked one of those two models. But when I tested all three tuned models on the real, untouched test set, the result flipped: both tree-based models landed around RMSE 33 on real test data — much worse than their cross-validation scores suggested — while Ridge's test RMSE (30.45) stayed close to what its own cross-validation score had predicted. The reason is that the cross-validation folds are all drawn from inside the training period, and tree-based models are flexible enough to fit small patterns of noise in each fold. That has nothing to do with how well they handle genuinely new, future data. Ridge's cross-validation and test scores matching each other is exactly what you'd expect from a model that has learned something real, rather than memorized noise.

That finding decided the final model choice: a tuned Ridge Regression model (alpha=10.0, with StandardScaler applied to the features) went into production for all three time windows, instead of either tree-based model.

6.3 Final validated performance

After I found and fixed a Feature Store data problem (the full story is in Section 9.3), I ran the entire pipeline again from scratch on clean data. These are the real, final results for the production Ridge models:

Horizon	RMSE	MAE	R^2
24h	22.36	17.56	0.785
48h	29.01	23.19	0.640
72h	30.45	24.68	0.604

Table 1 (repeated). Final test results for each forecast horizon.

Accuracy drops a little as the forecast window gets longer, which is what I expected. The 24-hour model gets a lot of help from strong, recent AQI lag features, which naturally matter less the further ahead you look — so the 72-hour model leans more on seasonal and calendar patterns instead.

6.4 Feature importance and explainability

month_cos — the cyclical feature for month — was by far the most important feature across every model I tried, making up about 58% of Random Forest's total feature importance on its own. That matches the earlier finding from the exploratory analysis: season affects Multan's AQI more than any short-term weather change. It's also a big reason Ridge did better than the tree-based models — a smooth, repeating pattern like this fits a straight-line model better than the step-by-step splits a tree naturally makes.

For explainability on the dashboard, I used SHAP (specifically shap.LinearExplainer, which works well with the linear Ridge model) to show which features drove each forecast. One thing worth explaining for anyone reading the SHAP results: several lag features are closely related to each other, so SHAP sometimes shows one lag with a positive effect and another with a negative effect, even though they're really capturing the same underlying signal. This is normal and expected given how closely related those features are — it isn't a bug. I added a short note on the dashboard explaining this so non-technical users don't misread it.

6.5 Model Registry

The three final Ridge models (aqi_ridge_24h, aqi_ridge_48h, aqi_ridge_72h) are stored in the Hopsworks Model Registry. Each one is saved as a single file containing the trained model, its StandardScaler, and the exact list of feature columns used during training. This way, the Streamlit app can never accidentally use a model with features that don't match what it was trained on.

7. Automation Pipeline

Two GitHub Actions workflows keep the whole system running without me having to manage a server:

- Feature pipeline (hourly): pulls the latest weather and air-quality data from Open-Meteo, builds the same 47 features described in Section 5, and updates the Hopsworks Feature Group. This is also what makes the self-healing target design work — every hourly run gets a chance to fill in real values for rows that used to have an empty target.
- Training pipeline (daily): reads the latest data from the Feature Store, retrains all three Ridge models, and saves the updated versions to the Model Registry, so the live forecasts stay current as new data comes in.

The Hopsworks API key is stored as a GitHub Secret, not anywhere in the code itself.

8. Dashboard & Deployment

The dashboard is a Streamlit app, hosted on Streamlit Community Cloud. I split the code into separate files inside a utils/ folder (hopsworks_utils.py, prediction.py, aqi_utils.py) instead of one long script, mainly so I could test the data access, the prediction logic, and the display logic separately as I built each part.

A few choices in the dashboard worth explaining:

- A warning banner appears whenever any forecast window goes above AQI 150, so someone glancing at the app gets a clear alert instead of having to read raw numbers

- The forecast part of the AQI chart is shown as a dotted line, different from the solid line used for past data — I didn't want the chart to look more certain about the future than the model actually is
- The status label reads 'STATION ACTIVE' instead of 'LIVE SYSTEM' — a small change in wording, but a more honest one, since the data updates every hour, not instantly
- Data and models are cached (@st.cache_data for one hour on features, @st.cache_resource for model files) to keep the app fast on Streamlit's free tier

The dashboard uses Space Grotesk for headings and IBM Plex for regular text — a clean, simple look that fits an air-quality tool better than a plain default font.

9. Challenges & Lessons Learned

A few real problems shaped this project more than any single design choice did. I'm writing about them in detail because they're an honest part of the internship, not just the parts that went smoothly.

9.1 Data source reliability

As explained in Section 3.1, AQICN/WAQI didn't work well for Multan — the location lookup pointing to a station in a different country was a clear sign I needed to change direction instead of trying to fix around it. Making that decision early, instead of spending days trying to patch unreliable station data, is probably the single choice that helped the rest of the project go smoothly.

9.2 The cross-validation leakage trap

This is covered in more detail in Section 6.2, but it's worth repeating here: normal time-series cross-validation still gave misleadingly good scores for both tree-based models. If I had trusted GridSearchCV's results alone, I would have shipped a Random Forest or XGBoost model that performed noticeably worse on real, new data. The only thing that caught this was checking every tuning result against a test set the models had never seen, and trusting that number over a better-looking cross-validation score.

9.3 The corrupted Feature Store partition

A small bug in my original feature engineering notebook meant a cleanup step ran, but its result was never actually saved. Because of this, 24 rows that didn't have enough history for lag features were stored in the Feature Store with empty values instead of being removed as intended. Actually fixing this took several tries on Hopsworks' free tier:

- A row-level delete (sending the bad rows with operation="delete") looked successful in the logs, but checking the data afterward showed the same 24 rows, unchanged, still sitting there.
- Repeated read timeouts, plus a background job stuck 'Initializing' for several hours, made this harder to figure out than it should have been. It eventually turned out to be caused by shared free-tier server load, not anything wrong in my own code.

- What actually worked was giving up on deleting rows in place. Instead, I read the current data, removed the bad rows myself in pandas, deleted the whole Feature Group, and recreated it fresh with data that was already clean. This avoided the unreliable delete process completely.

Because of this, the Feature Engineering notebook now has a permanent check for gaps in the hourly timeline, and a clear status check after every save to Hopsworks — so a similar problem would show up right away instead of sitting quietly in the data for weeks, like this one did.

9.4 Platform reliability on the free tier

Beyond that one issue, I ran into more than one case where Hopsworks reported a job as failed when it actually wasn't — including one case where the failure was just an unrelated timeout while the system was shutting down, even though the real data save had already finished successfully moments earlier. The lesson: on shared, free-tier infrastructure, a job's reported status is a hint, not the full truth. The only way to really confirm a save worked is to read the data back yourself and check it directly.

9.5 Local development environment (Windows)

A few Windows-only issues slowed down setup, noted here in case they help a future intern: hopsworks.login() needs a specific cert_folder setting, because its default assumes a Unix-style /tmp folder that doesn't exist on Windows; Hopsworks' internal Kafka connection always expects a /tmp folder too, even when not using streaming, so a matching D:\tmp folder has to be created by hand before every save; the API key needed every available permission, including Serving, or the program would crash from one missing permission; and I needed to install the hopsworks[python] package specifically, not the plain hopsworks package, to get every required piece.

10. Results Summary: Achieved vs. Planned

Checking against the eight parts listed in the original brief, all eight were completed:

- Feature pipeline — done, runs automatically every hour through GitHub Actions
- Historical backfill — done, about 3 years of hourly data
- Training pipeline with model comparison — done; Ridge, Random Forest, and XGBoost were all compared and tuned. An LSTM was considered but not built (Section 6.2), matching the brief's note that it was optional and only worth doing if it clearly helped
- Automation through GitHub Actions — done, an hourly feature pipeline and a daily training pipeline
- Streamlit dashboard — done, deployed on Streamlit Community Cloud
- Explainability and alerts — done, SHAP-based explanations plus a warning banner for AQI over 150
- Exploratory analysis before modeling — done, and it went further than originally planned, once the first outlier results didn't hold up to closer checking
- Final report — this document

The one real difference from the original plan is how much time went into fixing Feature Store and platform reliability problems (Sections 9.3–9.4) — something the original 20-day schedule didn't account for. Nothing planned had to be dropped because of it, but it's a fair account of where a good chunk of the time actually went.

11. Conclusion & Future Work

The Pearls AQI Predictor does what it set out to do: a free, serverless system that gives people in Multan a genuinely useful 3-day AQI outlook, built on a model I chose because it held up under honest testing, not because it had the best-looking cross-validation score. Just as important, this internship gave me hands-on experience with the kind of problem-solving a course doesn't usually teach — a data source that turned out to be unusable, a testing setup that quietly favored the wrong model, and a hosted platform whose own status messages couldn't always be trusted.

A few directions worth exploring if I keep working on this:

- Adding a second, independent AQI data source, if a reliable one for Multan becomes available, as a way to double-check the Open-Meteo numbers
- Trying an LSTM model now that there's a clean, gap-checked 3-year dataset to train on — skipping it earlier was mostly about time and data quality, not a final judgment that it wouldn't help
- Letting users set their own AQI alert level, since how sensitive people are to bad air quality really does vary from person to person
- Looking further into the unexplained low-AQI case from Feb 20 2024 (Section 4.2), ideally with upper-atmosphere weather data beyond what Open-Meteo's surface-level data can offer